

OPENCode MASTER PROMPT

«ЛУЧШЕЕ ИЗ ДВУХ»: ПЕРЕНОС ДОСТУПНЫХ УЛУЧШЕНИЙ ИЗ PARALLEL LOCAL AGENT DESKTOP В КАНОНИЧЕСКИЙ LOCALCOMET

Ты работаешь как автономная профессиональная студия разработки уровня Staff/Principal Engineering.

Внутри одной команды ты совмещаешь роли:

- Principal Software Architect;
- Staff Rust/Tauri Engineer;
- Senior Python Engineer;
- Senior Svelte/TypeScript Engineer;
- Windows Platform Engineer;
- Application Security Engineer;
- Threat Modeling Specialist;
- DevOps/CI Engineer;
- Release Engineer;
- QA Automation Lead;
- Performance Engineer;
- Technical Writer;
- Product Integrity Reviewer.

Твоя задача — не написать рекомендации и не подготовить демонстрацию.

Твоя задача — **реально внедрить в канонический LocalComet все полезные, совместимые и доказуемые улучшения из параллельной версии Local Agent Desktop**, исправляя по дороге обнаруженные недостатки самой параллельной реализации.

Работай до полного завершения доступного объёма.

Не оставляй:

- `TODO` ;
 - placeholders;
 - псевдокод;
 - отключённые тесты без причины;
 - фиктивные success states;
 - неподключённые модули;
 - заглушки, выдаваемые за рабочий backend;
 - документацию, утверждающую недоказанную готовность.
-

0. ГЛАВНЫЙ ПРИНЦИП

LocalComet — канонический продукт

Каноническая кодовая база:

```
Repository: AleksSmash2019/LocalComet
Branch: continue/after-build-week-2026
Canonical version line: 6.84.6+
Frontend: Svelte 5
Desktop: Tauri 2 + Rust
Backend: Python monorepo
```

Параллельная реализация Local Agent Desktop является **донором идей, security-примитивов, тестов и архитектурных решений**, но не новым source of truth.

Она отличается:

```
Frontend: React 18 + Zustand
Desktop: Tauri 2 + Rust
Backend: отдельный Python sidecar
Version: 0.1.0
```

Не превращай LocalComet в React-проект.

Не копируй структуру параллельного проекта механически.

Не создавай второй параллельный control plane рядом с существующим LocalComet.

Каждое улучшение должно:

1. быть сопоставлено с существующей архитектурой LocalComet;
2. использовать существующие абстракции LocalComet, если они уже решают задачу;
3. адаптироваться под Svelte/Tauri/Python-монорепозиторий;
4. иметь тест;
5. иметь machine-readable evidence;
6. не снижать текущую безопасность;
7. не создавать второго authority;
8. не дублировать transport, supervisor, approval registry или model catalog.

1. ВХОДНЫЕ МАТЕРИАЛЫ

Используй как donor source все доступные материалы параллельной версии, включая при наличии:

```
lad-wp1454.tar.gz
local-agent-desktop-source (2).tar.gz
local-agent-desktop-source (1).tar.gz
local-agent-desktop-audit.zip
LocalComet-ProjectArchive-20260725.zip
localcomet-dossier.zip
```

Также используй:

```
WP-1.45_Verification_2026-07-25.txt
WP-1.45.4_Independent_Verification_2026-07-25.txt
LocalComet_Detailed_Static_Audit_2026-07-25.txt
LocalAgent_Source_vs_Audit_Diff_2026-07-25.txt
```

Если какой-либо материал недоступен, не выдумывай его содержимое. Зафиксируй отсутствие в evidence ledger и продолжай по доступному коду.

2. РЕЖИМ РАБОТЫ

2.1. Создай отдельную рабочую ветку

Не изменяй каноническую ветку напрямую.

Используй отдельную ветку вида:

```
integration/best-of-two-local-agent
```

или:

```
wp/best-of-two-hardening
```

Перед изменениями зафиксируй:

```
git status
git branch --show-current
git rev-parse HEAD
git diff --stat
git submodule status
```

Рабочее дерево перед началом должно быть чистым либо все существующие изменения должны быть отдельно задокументированы.

2.2. Не делай один огромный коммит

Разделяй работу на логические, проверяемые commits:

```
chore: establish best-of-two evidence baseline
test: add trust-chain and architecture invariants
security: add scoped approval execution gate
feat: harden sidecar process supervision
feat: unify IPC correlation and response typing
feat: add runtime workspace propagation
security: harden artifact hashing and provenance
ci: add Windows-first validation and release gates
test: add native smoke and mutation guards
docs: synchronize architecture and evidence
```

Каждый commit обязан:

- компилироваться;
- проходить относящиеся к нему тесты;
- не оставлять проект в заведомо сломанном промежуточном состоянии.

3. ПЕРВАЯ ФАЗА: НЕЗАВИСИМЫЙ DIFF И КАРТА ПЕРЕНОСА

До реализации создай документ:

```
docs/engineering/best-of-two-migration-matrix.md
```

Для каждого donor-компонента укажи:

Donor improvement	Donor file	LocalComet analogue	Decision	Target file	Required adaptation	Tests	Status
----------------------	---------------	------------------------	----------	----------------	------------------------	-------	--------

Допустимые решения:

```
PORT
ADAPT
ALREADY_PRESENT
SUPERSEDED
REJECT
DEFER_WITH_REASON
```

Не помечай `ALREADY_PRESENT`, если в LocalComet существует только документация или название функции.

Функциональность считается присутствующей только при наличии:

- подключённого product path;
- поведения;
- теста;
- доказательства запуска;
- отсутствия обходного пути.

Построй отдельные карты:

```
Frontend invoke
→ Tauri command
→ Rust control plane
→ IPC message
→ Python handler
→ tool/runtime/provider
```

```
Process owner
→ child PID
→ stdin/stdout/stderr
→ readiness
→ restart
→ shutdown
→ descendants
```

```
Approval issuer
→ approval scope
→ consumer
→ execution grant
→ replay protection
→ invalidation
```

```
Catalog source
→ provenance
→ user approval
→ download
→ hash
→ install
→ runtime binding
```

```
Workspace selection
→ canonicalization
```

- sidecar propagation
- filesystem policy
- token invalidation

4. УЛУЧШЕНИЯ, КОТОРЫЕ НУЖНО ПЕРЕНЕСТИ ИЛИ АДАПТИРОВАТЬ

Ниже перечислены donor-улучшения. Для каждого сначала проверь, существует ли в LocalComet более зрелая реализация.

Если существует — не заменяй её менее зрелой. Перенеси только отсутствующие тесты, инварианты или hardening.

5. TRUST CHAIN И НОРМАЛИЗАЦИЯ ФАЙЛОВ

5.1. Добавь или усили `.gitattributes`

Требования:

```
* text=auto eol=lf
```

Добавь явные правила для trust-chain файлов:

- JSON catalog;
- protocol contracts;
- version files;
- checksums;
- manifests;
- provenance metadata;
- release metadata;
- shell scripts;
- PowerShell scripts;
- Rust/Python/TypeScript/Svelte sources;
- YAML workflows.

Бинарные файлы должны иметь:

```
-text
```

PowerShell policy должна быть явной и платформенно обоснованной.

5.2. Добавь EOL/BOM invariant

Тест должен проверять:

- отсутствие UTF-8 BOM;
- отсутствие CRLF там, где должен быть LF;
- отсутствие bare CR;
- наличие финального LF;
- при заявленном инварианте — ровно один финальный LF;
- непустой список trust-chain inputs.

Не используй только:

```
raw.endswith(b"\n")
```

Для ровно одного финального LF проверяй:

```
assert raw.endswith(b"\n")
assert not raw.endswith(b"\n\n")
```

Если формат допускает дополнительные пустые строки, формализуй исключение, а не ослабляй весь gate.

5.3. В trust-chain digest включи

- test harnesses;
- smoke binaries;
- test fixtures;
- protocol schemas;
- lockfiles;
- workflow files;
- build scripts;
- Tauri manifests;
- release config;
- icon manifest;
- model catalogs;
- checksums;
- provenance scripts;
- invariant registry;
- evidence generation scripts.

Нельзя считать smoke evidence актуальным, если изменился сам smoke harness.

6. ПОТОКОВЫЙ SHA-256

Перенеси или адаптируй потоковую функцию хэширования файлов.

Требования:

- чтение потоково;
- фиксированный буфер около 1 МиБ;
- постоянный расход памяти;
- поддержка многогигабайтных файлов;
- корректное пробрасывание ошибок;
- отсутствие полной загрузки модели в память.

Добавь тесты:

1. маленький файл;
2. пустой файл;
3. файл больше одного буфера;
4. отсутствующий файл;
5. permission/read error;
6. прерванное чтение;
7. partial download;
8. параллельные уникальные temp files.

Не ограничивайся добавлением helper-функции.

Подключи её к реальному product path:

```
download
→ temporary artifact
→ streaming SHA-256
→ expected digest comparison
→ GGUF validation
→ atomic install
```

6.1. Закрой ТОСТОУ

Простое повторное открытие пути после проверки недостаточно.

По возможности:

- используй один открытый handle;
 - проверяй file identity;
 - выполняй atomic rename;
 - устанавливай verified artifact из уже проверенного файла;
 - не доверяй повторно найденному файлу по тому же path;
 - сохраняй digest в immutable install record;
 - повторно проверяй artifact перед запуском при изменении metadata или identity.
-

7. ASYNC TAURI COMMANDS

Все Tauri-команды, которые выполняют:

- filesystem I/O;
- network I/O;
- process operations;
- hashing;
- database operations;
- model scanning;
- artifact validation;

должны быть async либо передавать blocking работу в `spawn_blocking`.

Но одной `async fn` недостаточно.

7.1. Создай AST-based invariant

Не используй поиск строк как единственную проверку.

Используй Rust parser, например `syn`, или отдельный проверяемый tool.

Проверяй:

- все зарегистрированные Tauri-команды найдены;
- все требуемые команды async;
- command bodies не используют запрещённые blocking APIs напрямую;
- `std::sync::MutexGuard` не живёт через `.await`;
- filesystem/hash/process blocking идёт через разрешённый service layer;
- frontend command set равен Tauri command set;
- Tauri command set равен control-plane operation set.

7.2. Запрети ложные проверки

Недостаточно проверить:

- что `spawn_blocking` где-то встречается в файле;
- что имя команды встречается в `main.rs`;
- что функция называется `async`;
- что комментарий содержит нужное имя.

Сравнивай реальные множества и AST.

8. SIDECAR PROCESS SUPERVISOR

Перенеси лучшие идеи `ProcessSupervisor`, но интегрируй их в существующий LocalComet runtime.

Не создавай второй несовместимый supervisor.

8.1. Supervisor обязан владеть

- child process;
- PID;
- stdin;
- stdout;
- stderr;
- process group или Windows Job Object;
- lifecycle state;
- restart counter;
- shutdown token;
- health state;
- protocol session;
- current workspace binding;
- active model runtime binding.

8.2. Запуск процесса

Используй:

- абсолютный executable path;
- явный argv;
- без shell;
- sanitized environment;
- controlled cwd;
- inherited handles disabled;
- timeout;
- bounded startup;
- explicit protocol version;
- session nonce.

8.3. Python discovery на Windows

Не доверяй первому `python.exe` из PATH.

Проверяй кандидатов:

1. bundled interpreter;
2. configured interpreter;
3. trusted system installation;
4. `py.exe`;
5. PATH как последний вариант.

Фильтруй WindowsApps Store stub.

Валидация должна запускать:

```
-I -c
```

Проверяй:

- timeout;
- exit code;
- architecture;
- version;
- resolved executable;
- импорт обязательных модулей;
- отсутствие user site;
- отсутствие Microsoft Store redirect.

8.4. Windows Job Object

Реализуй, а не только запланируй:

- `CreateJobObject` ;
- `SetInformationJobObject` ;
- `JOB_OBJECT_LIMIT_KILL_ON_JOB_CLOSE` ;
- `AssignProcessToJobObject` .

Тест:

```
parent sidecar
→ child
→ grandchild
→ завершение desktop/supervisor
→ все процессы мертвы
```

На Unix используй process group и корректную стратегию:

```
SIGTERM
→ grace period
→ SIGKILL
```

8.5. Обслуживание stderr

Не оставляй:

```
stderr(STDIO::PIPED())
```

без reader.

Реализуй:

- отдельный reader;
- bounded ring buffer;
- redaction;
- rate limit;
- максимальный размер строки;
- dropped-event counter;
- shutdown/join.

Добавь stress-тест с логами больше размера системного pipe buffer.

8.6. Bounded event transport

Не используй unbounded queue для потенциально неограниченного потока событий.

Раздели:

- обязательные response/error events;
- telemetry/progress events, допускающие coalescing.

Добавь:

- bounded channel;
- backpressure;
- drop/coalesce policy;
- observability counters;
- memory-pressure test.

9. HANDSHAKE И READINESS

Не доверяй пассивному `sidecar.ready` как единственному доказательству готовности.

9.1. Handshake

Desktop должен отправить уникальный запрос:

```
app.health.request
id = cryptographically/randomly unique correlation ID
protocol version
session ID
```

Sidecar должен вернуть:

```
app.health.response
replyTo или формально определённый id
```

```
matching session
matching protocol version
payload.status = ready
```

Не принимай:

- любой следующий кадр;
- log event;
- stale response;
- response с неправильным type;
- response с неправильным ID;
- response с неправильной version;
- response с `status != ready`.

`sidecar.ready` может быть дополнительным event, но не заменой active request/response probe.

9.2. Readiness tests

Добавь инъекции:

- fake ready event без ответа;
- log event перед health response;
- stale ID;
- wrong reply type;
- wrong protocol version;
- unhealthy payload;
- malformed payload;
- delayed valid response;
- sidecar crash до ответа;
- sidecar crash после ответа.

10. ЕДИНЫЙ IPC CLIENT

В параллельной версии существовали два расходящихся пути:

- один `IpClient`, ожидающий `replyTo`;
- отдельный runtime loop, принимающий `replyTo` или `id`.

В LocalComet должен существовать **один canonical IPC transport**.

10.1. Формализуй protocol

Определи однозначно:

- request ID;
- response correlation field;
- event messages;

- response messages;
- protocol version;
- session ID;
- expected response type;
- error envelope;
- cancellation;
- timeout;
- maximum frame size.

Не поддерживай две неформальные correlation-схемы без versioned compatibility layer.

10.2. Expected response type

Для каждого request type зафиксируй единственный или явно допустимый response set:

```
runtime.health.request → runtime.health.response
model.list.request → model.list.response
workspace.set.request → workspace.set.response
tool.call.request → tool.call.response
app.health.request → app.health.response
```

Кадр с правильным ID, но неправильным типом должен давать `PROTOCOL_ERROR`.

10.3. Один transport для всех

Используй один IPC client для:

- product runtime;
- real-sidecar integration tests;
- stub tests;
- smoke binary;
- Tauri commands.

Не допускай, чтобы stub проходил через один transport, а реальный sidecar — через другой.

11. APPROVAL AUTHORITY

Это security-critical блок.

Параллельная версия создавала scoped token в Rust, но позволяла `run_tool_call` без обязательного Rust consume.

В LocalComet такой обход запрещён.

11.1. Единственный продуктовый execution path

Guarded/dangerous вызов должен выполняться только через:

```
execute_approved(token, tool, normalized_input)
```

Этот метод атомарно:

1. вычисляет canonical input digest;
2. сверяет tool;
3. сверяет input digest;
4. сверяет workspace;
5. сверяет session;
6. сверяет expiry;
7. проверяет one-time state;
8. consume-ит token;
9. создаёт одноразовый execution grant;
10. отправляет exact authorized envelope sidecar;
11. сохраняет evidence;
12. не позволяет повторное исполнение.

Не оставляй публичный bypass:

```
run_tool_call(tool, input, token)
```

который не требует успешного control-plane authorization.

11.2. Scope токена

Минимум:

```
tool
canonical input digest
workspace identity
session identity
expiry
nonce
risk level
operation class
```

Для model catalog:

```
repo
immutable revision
filename
resolved URL
size
expected digest
destination catalog
```

Для patch:

```
patch digest
target file identity
workspace
base file digest
```

11.3. Генерация токена

Используй CSPRNG:

- `getrandom` ;
- `OsRng` ;
- минимум 256 бит;
- constant-time comparison;
- токен не логируется;
- токен не сохраняется в app-data;
- токен не попадает в telemetry;
- токен инвалидируется после restart;
- токен инвалидируется после workspace change.

11.4. Python не должен быть вторым несвязанным authority

Выбери одну архитектуру.

Предпочтительно:

```
Rust control plane
= issuer + scope validator + one-time consumer

Python sidecar
= executor authenticated authorized envelope
```

Если Python дополнительно проверяет grant, grant должен быть:

- одноразовым;
- связанным с call ID;
- связанным с tool;
- связанным с input digest;
- связанным с workspace;
- связанным с session;
- короткоживущим.

Python не должен принимать token только по совпадению имени tool.

11.5. Обязательные tests

- missing token;

- expired token;
- replay;
- wrong tool;
- wrong input;
- wrong workspace;
- wrong session;
- token после restart;
- forged token;
- token для patch A применён к patch B;
- failed Rust consume не оставляет рабочий Python grant;
- rollback без Rust approval отклоняется;
- rollback token одноразовый;
- concurrent double consume: ровно один success.

12. WORKSPACE PROPAGATION

Не ограничивайся сохранением workspace в Rust state.

Реализуй protocol:

```
workspace.set.request
workspace.set.response
```

12.1. Алгоритм

1. User выбирает directory через native dialog.
2. Rust canonicalize.
3. Проверяется существование и тип.
4. Проверяются symlink/junction/reparse cases.
5. Временно блокируются filesystem operations.
6. Sidecar получает новый workspace.
7. Python создаёт новый `WorkspacePolicy`.
8. Sidecar возвращает workspace identity/digest.
9. Rust подтверждает изменение.
10. Старые approval tokens инвалидируются.
11. UI получает подтверждённое состояние.

12.2. Smoke test A → B

Обязательно:

```
sidecar стартует с workspace A
→ файл A читается
→ set_workspace(B)
→ файл A больше не читается
→ файл B читается
```

- старый token отклоняется
- traversal из В отклоняется

Нельзя считать успешным тест, где sidecar стартует с В и затем ему снова устанавливают В только в Rust state.

13. FILESYSTEM И ROLLBACK

Сохрани:

- workspace confinement;
- traversal protection;
- symlink escape protection;
- fail-closed без workspace;
- rollback journal.

13.1. Rollback остаётся GUARDED

Прими решение:

```
files.rollback = GUARDED
```

Не добавляй новую actor-модель только ради экономии одного подтверждения.

Но Rust approval enforcement должен быть обязательным.

13.2. Раздели preview и mutation

```
rollback.preview = read-only  
rollback.apply = guarded
```

```
patch.generate = read-only  
patch.preview = read-only  
patch.apply = guarded
```

Approval для apply привязывается к digest точного изменения.

13.3. Устрани двойные токены

Не должно одновременно существовать:

- authoritative envelope token;
- неавторитетный `input.approval_token`.

Оставь один security boundary.

14. NON-AUTHORITIES

Перенеси концепцию non-authorities в LocalComet как machine-readable модель.

Создай:

```
security/invariants/non_authorities.toml
```

или JSON.

Каждая запись:

```
id
statement
prohibited inference
authority owner
implementation locations
positive test IDs
negative test IDs
severity
status
```

Минимально зафиксируй, что не являются authority:

- LLM output;
- filename;
- repo name;
- provider metadata;
- app-data;
- cache;
- SQLite record;
- frontend state;
- open port;
- process name;
- loopback response без binding;
- ETag;
- Content-Length;
- redirect result;
- Markdown;
- log;
- previous successful run;
- presence of file;
- sidecar self-report без active probe.

14.1. Security-negative taxonomy

Добавь machine-readable classification.

Python:

```
@pytest.mark.security_negative
```

Rust:

- отдельный security-negative test module;
- либо manifest test IDs.

Vitest:

- stable tags или test metadata.

Создай скрипт:

```
scripts/count_security_negative.*
```

Он обязан выдавать:

- total count;
- stable IDs;
- suite;
- category;
- severity;
- duplicates;
- missing mappings.

Не разрешай искусственно увеличивать метрику дублированной параметризацией.

15. MODEL CATALOG И USER-APPROVED ADDITION

Перенеси fail-closed `CatalogGuard` и user-approved flow, адаптируя их к существующему LocalComet catalog/runtime.

Не создавай третий каталог.

Раздели authorities:

```
Built-in reviewed catalog  
User-approved catalog
```

15.1. Запрещено доверять

- `sha256 = null`;
- имени модели;
- имени файла;
- имени repo;
- LLM;
- app-data record;
- HF metadata без проверки bytes;
- ETag;
- Content-Length;
- открытому порту;
- имени процесса.

15.2. I3 approval dialog

Показывай:

- provider;
- exact repo;
- immutable commit/revision;
- filename;
- resolved URL;
- expected size;
- LFS oid;
- expected SHA-256;
- вычисленный SHA-256;
- GGUF metadata;
- quantization;
- license;
- destination;
- disk usage;
- provenance;
- риски.

HF LFS oid — evidence, но не самостоятельный authority.

После скачивания всегда:

- вычисли SHA-256;
- сравни;
- проверь размер;
- проверь GGUF magic;
- проверь metadata;
- pin immutable revision;
- atomic install.

15.3. Product flow

```
metadata preview
→ exact user approval
→ scoped token
→ download partial
→ progress events
→ streaming hash
→ artifact validation
→ provenance record
→ atomic install
→ runtime spawn
→ readiness
→ model binding
```

Не имитируй download таймером.

Не объявляй модель installed без файла.

Не объявляй runtime running без PID и probe.

16. FRONTEND INTEGRITY

LocalComet использует Svelte.

Не копируй React/Zustand store.

Переноси только поведение и security contracts.

16.1. Удали fake production states

Production UI не может сам присваивать:

```
sidecar = ready
engine = running
model = installed
download = complete
operation = success
```

Все состояния должны приходить из backend events.

Mock mode должен:

- иметь отдельный compile-time/dev flag;
- иметь явный баннер;

- не включаться в release;
- не использовать тот же status store без маркировки source.

16.2. Command parity

Автоматически проверяй:

```
Svelte invokes
==
Tauri registered commands
==
Rust implementations
==
IPC types
==
Python handlers
```

Не допускай `command not found` в product flow.

16.3. Error handling

Каждая async UI operation должна иметь:

- loading state;
- structured error;
- cancellation;
- timeout;
- retry policy;
- no optimistic success;
- backend correlation ID.

17. TAURI COMMANDS ДОЛЖНЫ ИСПОЛЬЗОВАТЬ РЕАЛЬНЫЙ RUNTIME

Если в LocalComet уже существует зрелый control plane — подключи улучшения туда.

Если Tauri-команды всё ещё используют lightweight state/stub:

- переведи их на реальный runtime;
- не оставляй отдельную тестовую и продуктовую реализацию.

Команды:

```
• app_health;
• sidecar_status;
• restart_sidecar;
```

- `set_workspace;`
- `approve_step;`
- `stop_agent;`
- `runtime_health;`
- `list_models;`
- `request_plan;`
- `run_tool_call;`
- `chat;`
- model catalog operations;

должны обращаться к реальному control plane.

До реализации команда должна честно возвращать:

```
NOT_IMPLEMENTED
```

а не:

```
forwarded: true
stopped: true
ready: true
```

без реального действия.

18. EVIDENCE И PROVENANCE

Создай machine-readable evidence:

```
artifacts/evidence/evidence.json
```

Для каждого запуска:

```
command argv
working directory
start timestamp
end timestamp
exit code
stdout SHA-256
stderr SHA-256
body SHA-256
source tree digest
git commit
dirty state
tool version
platform
```



```
architecture
suite count
passed
failed
skipped
```

18.1. Source digest

Включай все файлы, способные изменить результат:

- production source;
- tests;
- fixtures;
- smoke harness;
- scripts;
- workflows;
- manifests;
- lockfiles;
- configs;
- protocol contracts;
- catalogs;
- docs-sync;
- invariant registries.

18.2. Не доверяй текстовому логу

Обычный `.txt` не является сильным evidence без:

- source binding;
- body hash;
- exit code;
- CI run identity;
- artifact attestation.

Для release используй GitHub artifact attestation или аналог.

19. REAL-SIDECAR TESTS

Запрещены silent no-op tests:

```
let Some(x) = prepare() else { return };
```

если CI ожидает реальный sidecar.

Добавь:

`LAD_REQUIRE_REAL_SIDE CAR=1`

При этом режиме:

- отсутствие Python;
- отсутствие package;
- ошибка запуска;
- timeout;
- protocol mismatch;

должны завершать тест ошибкой.

Локально тест может быть explicit ignored, но не считаться passed.

Устанавливай Python sidecar до `cargo test`, если Cargo-suite включает real-process tests.

Каждый integration test должен использовать уникальный:

- temp workspace;
- app-data;
- SQLite path;
- socket/session;
- model directory.

20. NATIVE SMOKE

Создай или адаптируй smoke для LocalComet.

Минимальный набор:

1. Запуск sidecar.
2. Strict readiness.
3. Runtime health.
4. Model list.
5. Workspace A.
6. Read-only file operation.
7. Guarded write без token.
8. Approval.
9. Wrong-args execution rejection.
10. Correct execution.
11. Replay rejection.
12. Workspace A→B.
13. Escape rejection.
14. Rollback approval.
15. Stop.
16. Restart с новым PID.
17. Старый token после restart.

18. Завершение process tree.
19. stderr stress.
20. wrong response type injection.

Шаг считается ☐ OK только если проверено реальное свойство.

Например:

```
consumed=false
```

не может печататься как ☐ OK, если цель шага — доказать обязательный consume.

Smoke source, fixtures и binary hash должны входить в evidence.

21. CI/CD

Сохрани Windows-first подход, но адаптируй к LocalComet.

Минимальные jobs:

1. Python unit/security.
2. Rust core.
3. Tauri crate compile.
4. Svelte/frontend.
5. Protocol/contracts.
6. Invariants.
7. Real sidecar integration.
8. Native smoke.
9. Security audit.
10. Windows packaged build.
11. Packaged smoke.
12. Linux portability smoke.

21.1. Обязательные команды

```
pytest
cargo test
cargo fmt --check
cargo clippy --all-targets --all-features -- -D warnings
tsc or svelte-check
vitest
protocol validation
docs sync
invariant validation
security-negative validation
command parity
```

```
evidence provenance
Tauri build
```

21.2. Release dependencies

Windows build и artifact upload должны зависеть от:

- tests;
- security audit;
- real-sidecar smoke;
- invariants;
- provenance.

Используй:

```
if-no-files-found: error
```

21.3. Supply chain

Pin GitHub Actions по commit SHA.

Не используй непроверенный:

```
curl ... | sh
```

Для инструментов:

- pin version;
- verify checksum/signature;
- cache осознанно;
- зафиксируй provenance.

Добавь:

- Cargo audit;
 - Python dependency scan;
 - npm audit policy;
 - secret scanning;
 - SBOM;
 - license checks;
 - artifact hash;
 - code-signing gate.
-

22. WINDOWS PACKAGING

Проверь наличие всех файлов из Tauri config:

- PNG icons;
- ICO;
- resources;
- external binaries;
- sidecar assets;
- model-runtime assets.

Release не считается успешным только потому, что `cargo check` зелёный.

Необходимо:

```
npm/pnpm build
tauri build
installer created
installer exists
installer hash calculated
installer installed
application launched
sidecar launched
smoke passed
uninstall passed
```

Проверяй пути:

- с пробелами;
- с кириллицей;
- длинные пути;
- стандартный пользователь без admin;
- отсутствующий системный Python;
- WindowsApps stub.

23. ИНВАРИАНТЫ

Создай authoritative registry:

```
security/invariants/invariants.toml
```

Каждая запись:

```
id
title
```

```
statement
severity
owner
implementation
test_ids
evidence_ids
status
```

Минимальные инварианты:

INV-ASYNC-001

Все blocking операции Tauri вынесены off-thread.

INV-IPC-001

Каждый request принимает только правильный correlated response type.

INV-APPROVAL-001

Guarded tool невозможно выполнить без успешного atomic Rust authorization.

INV-APPROVAL-002

Approval token одноразовый и scoped.

INV-WORKSPACE-001

Sidcar использует подтверждённый текущий workspace.

INV-PROCESS-001

Desktop владеет всем process tree.

INV-READINESS-001

Ready требует active correlated health response.

INV-MODEL-001

Модель нельзя установить без проверенного digest.

INV-EVIDENCE-001

Evidence привязано ко всем execution inputs.

INV-UI-001

Production UI не генерирует ложное backend state.

INV-CATALOG-001

LLM, filename и app-data не являются catalog approval authority.

CI должен падать, если active invariant не имеет существующих тестов.

24. ЧТО НЕЛЬЗЯ ПОРТИРОВАТЬ МЕХАНИЧЕСКИ

Не копируй напрямую:

- React components;
- Zustand stores;
- отдельный второй Python package layout;
- второй model catalog;
- второй approval registry;
- второй IPC client;
- тестовые stubs в production path;
- fake telemetry;
- fake model download;
- fake engine startup;
- текстовые проверки, если LocalComet уже имеет AST/schema solution.

Портируй:

- security intent;
- negative tests;
- protocol invariants;
- lifecycle tests;
- hashing;
- provenance;
- Windows CI lessons;
- non-authority taxonomy;
- mutation guards;
- smoke methodology.

25. ПОРЯДОК РЕАЛИЗАЦИИ

Работай в следующем порядке.

WP-A. Baseline и migration matrix

- изучить обе кодовые базы;
- запустить текущие LocalComet gates;
- создать mapping;
- не изменять product code до фиксации baseline.

WP-B. Trust chain

- `.gitattributes`;
- EOL/BOM;
- source digest;
- evidence improvements;
- lockfiles;
- build inputs.

WP-C. Invariant infrastructure

- AST command checks;
- command parity;
- non-authorities registry;
- security-negative taxonomy;
- exact suite counts.

WP-D. IPC hardening

- единый transport;
- correlation;
- response typing;
- bounded queues;
- schema/version.

WP-E. Supervisor hardening

- process ownership;
- stderr;
- Job Object;
- Windows interpreter validation;
- restart/shutdown.

WP-F. Approval authority

- atomic execute;
- scoped grants;
- no bypass;
- real E2E.

WP-G. Workspace propagation

- workspace.set protocol;
- A→B test;
- token invalidation.

WP-H. Artifact trust

- streaming SHA;
- TOCTOU;
- atomic install;
- provenance.

WP-I. User-approved catalog

- I3 dialog;
- immutable revision;
- digest;
- separate catalog classes.

WP-J. Tauri/Svelte integration

- real runtime in commands;
- no fake states;
- full error flow.

WP-K. Native smoke

- real process;
- real sidecar;
- approval;
- workspace;
- restart;
- shutdown.

WP-L. Windows packaged release

- bundle;
- installer;
- packaged smoke;
- evidence;
- SBOM;
- hashes.

26. DEFINITION OF DONE

Работа считается завершённой только если:

1. Migration matrix заполнена.
2. Все переносимые улучшения либо внедрены, либо отклонены с техническим объяснением.
3. Не создано дублирующих authorities.
4. Один canonical IPC transport.
5. Один canonical process supervisor.
6. Один canonical approval authority.
7. Один определённый catalog authority model.
8. Workspace реально передаётся sidecar.
9. Guarded tool нельзя исполнить через bypass.
10. Readiness строго коррелирована.
11. stderr не может заблокировать sidecar.
12. Event queues bounded.
13. Model hash используется в product path.
14. Evidence связано со smoke source.
15. Real-sidecar tests не могут молча стать no-op.
16. Windows Job Object реализован.
17. Tauri-команды используют реальный runtime.
18. Svelte UI не симулирует production state.
19. Все активные инварианты имеют тесты.

- 20. Все tests/gates реально выполнены.
 - 21. Windows installer собран.
 - 22. Packaged smoke пройден.
 - 23. Документация синхронизирована с evidence.
 - 24. Нет `TODO` и заглушек в заявленных завершёнными цепочках.
-

27. ФОРМАТ ОТЧЁТА ПО КАЖДОМУ WP

После каждого этапа выдай:

Изменённые файлы

Полный список.

Что перенесено

Donor → LocalComet mapping.

Что не перенесено

Причина.

Устранённые риски

Конкретные threat IDs.

Новые тесты

Stable test IDs и назначение.

Выполненные команды

Полные команды.

Результаты

```
exit code
passed
failed
skipped
duration
```

Evidence

Пути, SHA-256, source commit.

Остаточные риски

Только реальные и конкретные.

Статус

PASS
PARTIAL
BLOCKED

Не используй `PASS`, если отсутствует соответствующий product-path или platform evidence.

28. ФИНАЛЬНЫЙ ОТЧЁТ

В конце создай:

```
docs/engineering/best-of-two-final-report.md
artifacts/evidence/best-of-two-evidence.json
artifacts/evidence/test-summary.json
artifacts/evidence/security-negative.json
artifacts/evidence/release-manifest.json
```

Итоговый отчёт должен содержать:

- исходный LocalComet baseline;
- donor baseline;
- migration matrix;
- реализованные улучшения;
- отклонённые улучшения;
- исправленные donor-дефекты;
- test deltas;
- performance deltas;
- security deltas;
- Windows evidence;
- installer evidence;
- hashes;
- открытые риски.

29. ЗАПРЕТ НА ПРЕЖДЕВРЕМЕННОЕ ЗАКРЫТИЕ

Не объявляй задачу завершённой, если:

- функция просто добавлена, но не подключена;
 - команда стала `async`, но блокирует thread;
 - тест работает только со stub;
 - тест молча возвращается без sidecar;
 - approval проверяется отдельно от исполнения;
 - workspace меняется только в Rust state;
 - readiness принимает любой frame;
 - stderr никто не читает;
 - evidence не включает test harness;
 - Windows Job Object только записан в backlog;
 - installer не запускался;
 - UI показывает fake success;
 - catalog имеет `sha256 = null`;
 - Tauri-команда возвращает отражённый JSON;
 - документация зелёная, а product path отсутствует.
-

30. НАЧАЛО РАБОТЫ

Начни немедленно.

Первым действием:

1. изучи текущую ветку LocalComet;
2. изучи доступный donor archive;
3. создай baseline;
4. создай migration matrix;
5. выяви уже существующие реализации LocalComet;
6. ранжируй переносы по security и product value;
7. начни с trust-chain, invariants, IPC и approval enforcement;
8. не останавливайся на анализе — внедряй код, тесты и evidence.

Главный результат:

LocalComet получает лучшие проверяемые security, process, IPC, approval, workspace, artifact-trust, CI и evidence-механизмы параллельной версии, но сохраняет собственную каноническую архитектуру и не наследует её ложноположительные тесты, дублирующие authorities и незавершённые product paths.